

Jurnal Praktikum

Sistem Operasi

Modul 6: Proses Sekuensial dan Konkuren

Tujuan:

1. Mahasiswa mampu membuat proses berjalan secara sekuensial.
2. Mahasiswa mampu membuat proses berjalan secara konkuren.

Catatan:

1. Praktikan wajib untuk screenshot setiap langkah yang dikerjakan hingga tampilan output akhir.
2. Untuk soal source code, kumpulkan SS-nya saja.
3. Praktikan wajib untuk melakukan screenshot lengkap dengan nama root. Contoh: root@username.
4. Berikan identitas nama - nim dalam bentuk comment di Source Code.
5. Harap kerjakan secara mandiri, jika tidak paham silahkan bertanya kepada Asisten Praktikum masing-masing. Dilarang mengcopy jawaban dan source code dari teman!

Jurnal

1. **[10 Poin]** Selain hardware (memory), batasan maksimal proses dapat ditentukan dengan secara software. Pada Linux maksimal proses adalah 4194303 proses (64 bit) dan 32767 proses (32 bit) dapat dilihat melalui perintah `$cat /proc/sys/kernel/pid_max`

Carilah pada source code Xinu yang memberi batasan mengenai banyaknya proses yang bisa dibuat! Berapa maksimal proses dalam Xinu? Ubah menjadi maksimal 150 proses!

Jawaban:

Maksimal 8 proses dalam Xinu.

```
include > C process.h > ...
1  /* process.h - isbadpid */
2
3  /* Maximum number of processes in the system */
4
5  #ifndef NPROC
6  #define NPROC 8
7  #endif
```

Diubah menjadi 150 proses

```
include > C process.h > NPROC
1  /* process.h - isbadpid */
2
3  /* Maximum number of processes in the system */
4
5  #ifndef NPROC
6  #define NPROC 150
7  #endif
```

2. [20 Poin] Jalankan kode sekuensial!

Jawaban:

```
system > C main.c > main(void)
1  /* main.c - main */
2
3  #include <xinu.h>
4
5  void A() {
6      kprintf("Proses A jalan\n");
7  }
8  void B() {
9      kprintf("Proses B jalan\n");
10 }
11 process main (void)
12 {
13     kprintf("=== SEKUESIAL ===");
14     A();
15     B();
16     return OK;
17 }
18
```

Output:

```
Obtained IP address 10.0.3.15 (0x0a00030f)
=== SEKUESIAL ===
Proses A jalan
Proses B jalan
█
```

3. [20 Poin] Jalankan kode konkuren!

Jawaban:

```
1  /* main.c - main */
2
3  #include <xinu.h>
4
5  process A() {
6      kprintf("Proses A jalan\n");
7      return OK;
8  }
9  process B() {
10     kprintf("Proses B jalan\n");
11     return OK;
12 }
13 process main (void)
14 {
15     kprintf("=== Konkuren ===\n");
16     resume(create(A, 1024, 20, "A", 0));
17     resume(create(B, 1024, 20, "B", 0));
18     sleepms(500);
19     return OK;
20 }
```

Output:

```
Obtained IP address 10.0.3.15 (0x0a00030f)
=== Konkuren ===
Proses A jalan
Proses B jalan
```

4. [50 Poin] Buatlah 2 **proses** produser dan konsumer. Produser memproduksi angka integer dari 1-1000. Konsumer mengkonsumsi integer yang diproduksi oleh produser dan menampilkannya! (Gunakan variabel global bertipe int32 bernama n yang digunakan secara bersama oleh kedua proses)

```

int32 n = 0; //global variabel

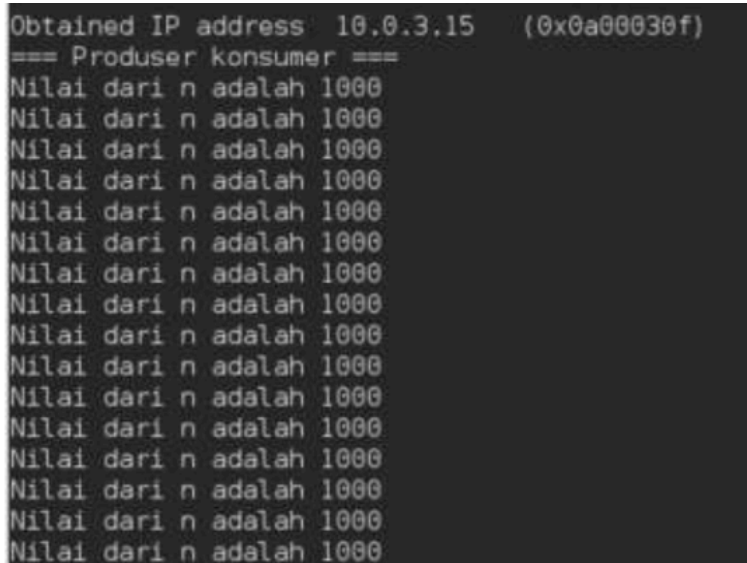
void produser(void){
    int32 i;
    for (i=1; i<=1000; i++){
        n++;
    }
}

void konsumen(void){
    int32 i;
    for (i=1; i<=1000; i++){
        printf("Nilai dari n adalah %d\n",n);
    }
}

```

Hasil dari program ini cukup mengejutkan (tidak akan sesuai dengan intuisi awal).
Jelaskan mengapa hasilnya seperti itu!

Jawaban:



```

Obtained IP address 10.0.3.15 (0x0a00030f)
=== Produser konsumen ===
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000

```

Hasil program tidak sesuai intuisi karena proses produser dan konsumen berjalan secara konkuren dan mengakses variabel global n secara bersama-sama tanpa sinkronisasi. Akibatnya, konsumen dapat membaca nilai n pada kondisi yang berbeda-beda, baik sebelum, saat, maupun sesudah. Fenomena ini disebut race condition.